Core Evidence 5: Preprocessing & Feature Engineering Log

IT3091 Machine Learning — Industry Explorer Track

Group ID: `2026-AI-36`

Project: ChefCast AI — Restaurant Revenue & Product Demand Forecasting

Version: Final Submission — October 2026

> *This log documents every preprocessing and feature engineering step with explicit justification for each choice. Per the IT3091 rubric, preprocessing decisions must demonstrate evidence-based reasoning, not arbitrary convention.*

1. Preprocessing Pipeline Architecture

1.1 Stage 1 — String Normalization & Date Parsing

```
Input: pos_raw_dataset.csv (127,093 rows) Steps: 1. Strip leading/trailing whitespace from all string fields 2. Enforce uppercase standardization on ItemName (e.g., "Lunch" → "LUNCH"; prevents split inventory counts) 3. Convert INVDATE to ISO 8601 format (YYYY-MM-DD) 4. Parse date as pandas datetime64 for temporal operations Justification: String normalization prevents the same dish being counted as multiple distinct items due to case or spacing differences.
```

1.2 Stage 2 — Chronological Validity Filter

```
Filter: INVDATE >= 2024-03-27 Rows removed: 147 (pre-commissioning system records) Justification (EDA Insight 2.1): Records dated 2009–2020 represent POS terminal commissioning tests, not authentic commercial transactions. Retaining them introduces ghost revenue signals in early lags.
```

1.3 Stage 3 — Transaction Quality Filters

```
Filter A: Qty > 0 (exclude voids and returns) Rows removed: 931 Justification: Negative quantities are system reversal entries for cancelled orders. They do not represent consumable demand. Filter B: UnitPrice > 0 (exclude promotional adjustments) Rows removed: 72 Justification: Negative prices are discount line items appended to invoices — not actual food items with a selling price. Filter C: UnitPrice ≤ 20,000 LKR AND Qty ≤ 300 Rows removed: 52 Justification: Maximum realistic dish price in Sri Lankan casual dining is ~1,500 LKR. Upper bound of 20,000 LKR provides a generous buffer for catering orders. Qty > 300 units in a single line item is inconsistent with any authentic restaurant transaction. Result: 125,912 valid transactions retained (99.1% of original)
```

1.4 Stage 4 — Product Scope Filtering

```
Filter: ItemName IN (11 selected fresh handmade items) Items: LUNCH, BREAKFAST, CHICKEN RICE FULL, CHICKEN CURRY, TEA, ROLLS, PATIS, SAMOSA, DATE CAKE, BUTTER CAKE, WATALAPPAN Justification (DEC-03): 5-criterion framework — kitchen-prepared, high revenue share, high frequency, perishable, complete historical coverage.
```

1.5 Stage 5 — Revenue Line Calculation

```
LineRevenue = UnitPrice × Qty (per transaction row) Justification: Standard retail revenue calculation. Unit prices are item-day specific; volume × price = turnover contribution.
```

1.6 Stage 6 — Temporal Aggregation

```
Lens 1: GROUP BY Date → SUM(LineRevenue) → daily_revenue_series.csv Output: 895-row time series [Date, Revenue] Lens 2: GROUP BY Date, ItemName → SUM(Qty) → daily_product_demand_long.csv Output: 9,845-row panel [Date, ItemName, Qty] (895 days × 11 items = 9,845 panel cells; some days/items may be zero)
```

1.7 Stage 7 — Calendar Reindexing & Zero-Filling

```
Operation: Reindex from 2024-03-27 to 2026-09-07 (895 continuous days) Insert 0.0 for any date with no qualifying POS records Justification (DEC-05): Dropping zero-sales dates breaks the mathematical continuity of lag_7 and lag_14 features. If Thursday 2026-04-10 is dropped, then lag_7 for Thursday 2026-04-17 would incorrectly reference Wednesday. Zero-filling preserves the exact 7-day temporal structure while accurately representing that no kitchen sales were recorded on those dates. Note: "No qualifying POS sales recorded" ≠ confirmed restaurant closure.
```

---

2. Feature Engineering: Complete Feature Taxonomy

The Governing Principle: Strict Temporal Leakage Prevention (DEC-06)

Definition of Data Leakage (Time-Series Context):

A feature causes lookahead leakage if it encodes information about day ***t***'s own outcome — i.e., if computing the feature requires knowing ***y_t*** before making a prediction for ***y_t***.

Enforcement Mechanism:

All rolling statistics and lag features are computed on a series explicitly shifted by 1 position:

```
series_shifted = series.shift(1) # shift=1 ensures index t uses value from t-1
```

This means: at prediction time for day ***t***, the most recent revenue information the model can access is ***y_{t-1}*** (yesterday's actual sales). Day ***t***'s sales are unknown to the model — exactly as they would be at 06:00 AM in live production.

2.1 Category 1: Calendar & Temporal Features

Feature Name	Type	Derivation	Justification
`day_of_week`	Integer (0–6)	`datetime.dayofweek`	Monday=0, Sunday=6 weekly cycle
`day_of_month`	Integer (1–31)	`datetime.day`	Month-within-month patterns
`month`	Integer (1–12)	`datetime.month`	Annual seasonal cycle
`quarter`	Integer (1–4)	`datetime.quarter`	Quarterly demand cycles
`week_of_year`	Integer (1–52)	`datetime.isocalendar().week`	Annual week-level pattern
`is_weekend`	Binary (0/1)	`day_of_week >= 5`	Weekend surge indicator

Why include both integer and cyclical representations?

Integer `month` captures monotonic scale information (month 12 > month 6 in calendar order), while cyclical ***\sin/\cos*** encodings (Section 2.2) capture the circular wrap-around continuity. Both representations carry complementary information for tree-based and linear models respectively.

2.2 Category 2: Cyclical Trigonometric Encodings

The Problem with Linear Integer Encodings:

A linear model treating `day_of_week` as 0–6 perceives Sunday (6) as "6× further from Monday (0)" than Monday itself. In reality, Sunday and Monday are adjacent in the weekly cycle. This distortion causes linear models to underperform on cyclical seasonal data.

The Solution — Sine/Cosine Dual Encoding:

$***{sin_dow} = \sin\!\left(\frac{2\pi \cdot {day_of_week}}{7}\right), {cos_dow} = \cos\!\left(\frac{2\pi \cdot {day_of_week}}{7}\right)***$

$***{sin_month} = \sin\!\left(\frac{2\pi \cdot {month}}{12}\right), {cos_month} = \cos\!\left(\frac{2\pi \cdot {month}}{12}\right)***$

Proof of correctness: The Euclidean distance between ***(\sin_{dow}(6), \cos_{dow}(6))*** and ***(\sin_{dow}(0), \cos_{dow}(0))*** is exactly equal to the distance between adjacent days — preserving the circular weekly topology.

Feature	Values Range	Cycle Captured
`sin_dow`, `cos_dow`	[–1, +1]	7-day weekly cycle
`sin_month`, `cos_month`	[–1, +1]	12-month annual cycle

2.3 Category 3: Autoregressive Lag Features

Definition: $*{lag}_k(t) = y_{t-k}$ — the actual observed value k days prior to day t.

Feature	Lag Period	Primary Domain Meaning
`Revenue_lag_1` / `Quantity_lag_1`	$t - 1$	Immediate momentum (yesterday)
`Revenue_lag_2` / `Quantity_lag_2`	$t - 2$	2-day recent trend direction
`Revenue_lag_3` / `Quantity_lag_3`	$t - 3$	3-day short-cycle signal
`Revenue_lag_7` / `Quantity_lag_7`	$t - 7$	**Same weekday last week — strongest predictor**
`Revenue_lag_14` / `Quantity_lag_14`	$t - 14$	Same weekday two weeks ago (fortnightly cycle)
`Revenue_lag_21` / `Quantity_lag_21`	$t - 21$	Same weekday three weeks ago (monthly pattern)

Why lag_7 is the dominant predictor:

The EDA-confirmed weekly cyclicality means "this coming Friday will statistically resemble last Friday" more than any other single historical reference point. LightGBM's feature importance confirmed `Revenue_lag_7` as a top-5 contributor to Lens 1 predictions.

Burn-in effect: The maximum lag of 21 days means the first 21 rows of the feature matrix contain NaN values and are dropped. This is why the final dataset has 874 rows (895 – 21 burn-in days) rather than 895.

2.4 Category 4: Strictly Shifted Rolling Window Statistics

Formula (correctly shifted):

$$*{roll_mean_W}(t) = \frac{1}{W} \Sigma_{k=1}^{W} y_{t-k}*$$

$$*{roll_std_W}(t) = \sqrt{\frac{1}{W-1} \Sigma_{k=1}^{W} (y_{t-k} - {roll_mean_W}(t))^2}*$$

Where the series is shifted by 1 before applying `.rolling(W)`.

Feature	Window	Domain Interpretation
`roll_mean_7`	7 days	Smooth weekly demand baseline — removes single-day noise
`roll_std_7`	7 days	Weekly demand volatility — high std indicates upcoming uncertainty
`roll_mean_14`	14 days	Fortnightly smoothed baseline — captures bi-weekly patterns
`roll_mean_30`	30 days	Monthly trailing average — long-horizon baseline level
`roll_min_7`	7 days	Worst-case recent demand floor
`roll_max_7`	7 days	Best-case recent demand ceiling

2.5 Category 5: Relative Trend Velocity Ratios

$$*{ratio_lag1_to_roll7}(t) = \frac{y_{t-1}}{{roll_mean_7}(t) + \varepsilon}*$$

$$*{ratio_lag7_to_roll14}(t) = \frac{y_{t-7}}{{roll_mean_14}(t) + \varepsilon}*$$

Where $*\varepsilon = 1 \times 10^{-5}*$ prevents division-by-zero on zero-demand calendar dates.

Domain interpretation:

- ratio > 1.0: Recent demand is trending above the rolling baseline → kitchen should prepare extra.
- ratio < 1.0: Recent demand is decelerating below baseline → kitchen should adopt lean preparation.

These ratios provide dimensionless momentum signals that are particularly effective for LightGBM, which can create sharp partitions at ratio = 1.0.

2.6 Category 6: Product Entity One-Hot Encodings (Lens 2 Only)

Purpose: Enable a single shared panel model to learn individual dish baselines while sharing temporal lag information across all 11 products.

Feature	Encoding	Value
`item_LUNCH`	Binary	1 if this row is for LUNCH, else 0
`item_BREAKFAST`	Binary	1 if this row is for BREAKFAST, else 0
`item_CHICKEN_RICE_FULL`	Binary	1 if this row is for CHICKEN RICE FULL
`item_CHICKEN_CURRY`	Binary	1 if this row is for CHICKEN CURRY
`item_TEA`	Binary	1 if this row is for TEA
`item_ROLLS`	Binary	1 if this row is for ROLLS
`item_PATIS`	Binary	1 if this row is for PATIS
`item_SAMOSA`	Binary	1 if this row is for SAMOSA
`item_DATE_CAKE`	Binary	1 if this row is for DATE CAKE
`item_BUTTER_CAKE`	Binary	1 if this row is for BUTTER CAKE
`item_WATALAPPAN`	Binary	1 if this row is for WATALAPPAN

Why OHE rather than a fixed entity embedding?

For 11 items with sufficient training history, OHE provides maximum interpretability and avoids the implicit ordering artifact of integer label encoding (which would falsely imply LUNCH=0 < TEA=4 in a linear sense). Ridge Regression's L2 penalty appropriately handles the slight multicollinearity introduced by 11 binary indicators.

---

3. Final Feature Matrix Summary

Feature Matrix	Rows (after burn-in)	Feature Count	Target Variable	Lens
Lens 1 Revenue	874 days	30 features	`Revenue` (LKR/day)	Lens 1
Lens 2 Demand (long-format)	9,614 item-days (874 × 11)	41 features (30 + 11 OHE)	`Qty` (portions/day/item)	Lens 2

---

4. Preprocessing Decisions That Were Explicitly Rejected

Rejected Approach	Why Rejected	Consequence if Used
Unshifted rolling averages	Leaks day t's own value into predictor for day t	Model would appear near-perfect on validation but fail at zero-error in production
PCA for dimensionality reduction	Destroys temporal lag interpretability; 30 features is manageable for all candidates	Loss of feature importance explanability in the final report
External weather data imputation	No ground truth external weather API for Sri Lanka POS dates; invented features add noise	Spurious correlations with no generalization value
Integer label encoding for dish names	Implies ordinal rank (LUNCH > BREAKFAST numerically)	Linear models would treat dish names as magnitudes